Итоговый проект: «Веб-приложение полного цикла»

Общая идея

Это **проект**: вы сами придумываете тему, сами проектируете архитектуру и реализуете её с нуля. В проекте должны быть задействованы все технологии, пройдённые за курс: Razor Pages, AJAX, REST API, SignalR, EF Core и система безопасности (Identity, авторизация, JWT, защита от CSRF/XSS).

Срок: 2–3 недели.

Формат работы: индивидуально.

Что сдаётся: готовое приложение + короткая презентация (5 минут) с демонстрацией.

Выбор темы

Вы **сами выбираете** предметную область. Единственное условие — тема должна естественно вписывать все технологии, включая безопасность. Не нужно натягивать SignalR на проект, где он не нужен.

Подходящие темы (примеры для вдохновения, не для копирования):

- Аукцион в реальном времени: ставки, таймер, список лотов, роли продавец/покупатель
- Доска объявлений: подача объявления, комментарии, просмотры, чат с продавцом, модерация админом
- Система бронирования: расписание, занятость слотов, уведомления, роли клиент/админ
- Классный журнал: оценки, посещаемость, уведомления родителям, роли учитель/ученик/родитель
- Система опросов: создание опроса, голосование, live-результаты, автор только свои опросы
- Афиша событий: список мероприятий, бронирование мест, счётчик свободных мест, live-уведомления
- Интерактивная карта задач: точки на карте, добавление меток, обсуждение, роли

Тему нужно согласовать с преподавателем до начала работы.

Технологические требования

Проект должен использовать **все шесть блоков технологий**. Ниже — что именно нужно реализовать в каждом блоке. Эти требования обязательны: если хотя бы один блок не задействован — проект не будет сдан на ОТЛИЧНО.

Блок 1. Razor Pages — основа приложения

Razor Pages — это фундамент: страницы, маршрутизация, серверная обработка.

Что должно быть реализовано:

- **Минимум 5 страниц** с разными маршрутами. Например: главная, список элементов, детальная страница, форма создания, страница профиля или настроек.
- **Минимум 2 формы** с серверной валидацией (Data Annotations) и отображением ошибок на странице. Не AJAX — именно классическая отправка формы с перезагрузкой, чтобы показать, что вы умеете работать с серверной валидацией и ModelState.

- **Минимум 1 страница с серверным рендерингом данных** из базы: страница загружается, данные уже есть (не через AJAX, а через @model и Razor-синтаксис).
- **Layout-страница** с навигацией, общим хедером и футером. Все страницы используют общий layout. В навигации должны отображаться разные ссылки в зависимости от того, вошёл пользователь или нет, и от его роли.
- **Tag Helpers** для форм и ссылок: asp-for, asp-action, asp-route-id и т. д.

Что демонстрирует этот блок:
Вы умеете строить серверную часть приложения, рендерить данные на сервере, работать с валидацией и маршрутизацией.

Блок 2. AJAX — частичные обновления без перезагрузки

AJAX используется там, где перезагружать страницу неудобно или не нужно.

Что должно быть реализовано:
- **Минимум 3 интерактивных действия через AJAX** (Fetch API или XMLHttpRequest). Примеры: добавить в избранное, удалить комментарий, проголосовать, отметить как прочитанное, изменить статус, загрузить дополнительные элементы (пагинация без перезагрузки).
- **Минимум 1 действие с частичным обновлением DOM**: после AJAX-запроса страница не перезагружается, а обновляется только нужный фрагмент (элемент списка, счётчик, статус).
- **Обработка ошибок AJAX**: если запрос не удался — пользователь видит сообщение, а не молчание. Либо alert, либо блок на странице. Если пользователь не авторизован (401) — корректное сообщение, а не тихая ошибка.
- **Минимум 1 действие с обновлением данных в базе через AJAX**: не просто чтение, а реальное изменение состояния (статус, оценка, удаление).
- **Антифоргери-токен в AJAX-запросах**: все POST/PUT/DELETE AJAX-запросы к Razor Pages или MVC-контроллерам должны отправлять антифоргери-токен в заголовке X-CSRF-TOKEN. Токен берётся из <meta name="csrf-token"> на странице.

Что демонстрирует этот блок:
Вы умеете отправлять асинхронные запросы из браузера, обрабатывать ответы, обновлять интерфейс без перезагрузки страницы и защищать запросы от CSRF.

Блок 3. REST API — отдельный программный интерфейс

REST API — это не просто обработчики AJAX-запросов на странице. Это **отдельный, самостоятельный слой**, который можно вызвать через Postman, curl или другой клиент.

Что должно быть реализовано:
- **Минимум 1 контроллер API** (не Razor Page, а Web API-контроллер), доступный по маршруту /api/....
- **Полный CRUD для хотя бы одной сущности**: GET (список), GET по ID, POST (создание), PUT (обновление), DELETE (каскадное удаление). Все пять методов — обязательно.
- **Корректные HTTP-статусы**: 200, 201 (с заголовком Location), 204, 404, 400 (с ProblemDetails при невалидных данных).
- **DTO**: данные отдаются и принимаются через отдельные классы DTO, а не через Entity-модели напрямую.
- **Версионирование**: API доступен как минимум в двух версиях (/api/v1/... и /api/v2/...). Вторая версия должна содержать хотя бы одно осмысленное изменение контракта (добавлено поле, убрано поле, изменён тип).
- **JWT-аутентификация для API**: все эндпоинты API защищены через [Authorize] с JWT-схемой. Доступ без токена — 401 Unauthorized.

- **Login-эндпоинт**: POST /api/auth/login — принимает e-mail и пароль, проверяет через SignInManager.CheckPasswordSignInAsync, и если верно — выпускает JWT с claims (ID пользователя, e-mail, роли). Срок жизни токена — не более 1 часа.
- **Авторизация в API**: обычный пользователь через API видит и управляет только своими данными. Админ — всеми. UserId берётся из claims токена, а не из тела запроса.
- **Swagger**: API задокументирован, Swagger UI доступен и показывает все эндпоинты с описаниями. В Swagger настроена возможность вставить JWT-токен и тестировать защищённые эндпоинты.
- **Пример .http-файла**: файл с запросами ко всем эндпоинтам API, включая логин (получение токена) и вызовы с токеном, который можно запустить прямо из IDE.

Что демонстрирует этот блок:
Вы умеете проектировать отдельный программный интерфейс по правилам REST, работать с DTO, статус-кодами, версионированием, документацией и защищать API через JWT.

Блок 4. SignalR — обновления в реальном времени

SignalR используется там, где данные меняются на сервере, и клиент должен узнать об этом мгновенно.

Что должно должно быть реализовано:
- **Минимум 1 хаб** с реальной бизнес-логикой. Это должны быть уведомления, связанные с предметной областью проекта.
- **Минимум 2 типа событий**, которые сервер отправляет клиентам через SignalR. Примеры: новый комментарий к объявлению, изменение статуса задачи, новая ставка на аукционе, обновление счётчика просмотров, уведомление «пользователь присоединился».
- **Минимум 1 использование групп**: клиенты делятся на группы по какому-то признаку (проект, категория, комната, событие), и события отправляются только в нужную группу.
- **Обработка переподключения**: клиент настроен на автоматическое переподключение, и после восстановления связи заново вступает в нужные группы.
- **Минимум 1 уведомление через IHubContext**: событие отправляется не из хаба, а из другого места приложения (сервиса, обработчика, фонового процесса).
- **Жизненный цикл**: обработка подключения и отключения клиентов с осмысленными действиями (обновление счётчика онлайн, уведомление других, логирование).
- **Авторизация хаба**: хаб защищён атрибутом [Authorize]. Неавторизованный пользователь не может подключиться. В обработчиках OnConnectedAsync/OnDisconnectedAsync доступен Context.User — можно определить, кто именно подключился, и направлять его в нужные группы на основе его данных.

Что демонстрирует этот блок:
Вы умеете создавать хабы, работать с группами, жизненным циклом подключений, переподключениями, отправкой уведомлений из внешних сервисов и защищать хаб аутентификацией.

Блок 5. EF Core — база данных

Минимум 5 таблиц с осмысленными связями. Просто «добавить таблицу с одним полем» не принимается. Каждая таблица должна иметь чёткую бизнес-роль и участвовать в логике приложения.

Обязательные типы связей:
- **Минимум одна связь 1:M** (один ко многим): например, Проект → Задачи, Пользователь → Заказы.

- **Минимум одна связь M:N** (многие ко многим) — через отдельную связующую сущность с явной таблицей. Это важно, чтобы вы видели, как EF Core маппит связующую таблицу и как с ней работать.
- **Минимум 1 таблица с составным первичным ключом** (связующая сущность для M:N).

Конкретные требования:

- **Миграции**: база разворачивается одной командой dotnet ef database update. Никаких ручных SQL-скриптов или создания таблиц в СУБД.
- **Seed data**: при первом запуске в БД загружаются тестовые данные (минимум 3–5 записей на каждую таблицу). Seed должен быть в коде, а не в миграции.
- **Индексы**: минимум 2 индекса (помимо PK):
  - один на поле, по которому идёт частый поиск/фильтрация (например, Email, Title, Status);
  - второй — на поле, по которому сортируют списки (например, CreatedAt, Priority).
- **Конфигурация через Fluent API**: минимум 3 настройки (длина строки, точность decimal, каскадное удаление и т. п.) задаются не атрибутами, а через OnModelCreating.
- **Навигационные свойства**: у всех связанных сущностей есть навигационные свойства в обе стороны (где это логично).

Что демонстрирует этот блок:
Вы умеете проектировать схему базы данных, настраивать связи, индексы, seed-данные и миграции через EF Core.

Блок 6. Безопасность — аутентификация, авторизация, защита от атак

Это блок, который пронизывает весь проект. Безопасность — не отдельная фича, а требование к каждому действию.

6.1. Аутентификация через ASP.NET Core Identity

Что должно быть реализовано:

- Подключите ASP.NET Core Identity. Используйте встроенный IdentityUser или создайте свой AppUser : IdentityUser с дополнительными полями (например, FullName, AvatarUrl).
- Контекст базы данных наследуется от IdentityDbContext.
- Настройте Identity с параметрами: минимальная длина пароля — 6 символов, требование цифры — обязательно, блокировка после 5 неудачных попыток на 15 минут.
- Создайте страницы **Register** и **Login** — классические Razor Pages с серверной валидацией.
- После успешной регистрации — автоматический вход и редирект на главную.
- Создайте страницу **Logout** (POST, не GET — чтобы по ссылке нельзя было разлогинить).
- На главной странице: если пользователь вошёл — показывается его имя и кнопка «Выйти». Если нет — ссылки «Войти» и «Зарегистрироваться».

6.2. Роли и авторизация

Что должно быть реализовано:

- **Минимум 2 роли** в проекте (например, Admin и User, или Admin/Author/Reader). Роли создаются при запуске (seed), а не вручную.
- При регистрации новому пользователю автоматически назначается базовая роль (например, User).
- Создайте seed-пользователя-админа: при первом запуске в базу добавляется пользователь с заранее заданными логином и паролем и ролью Admin. Проверьте: при повторном запуске он не дублируется.
- **Минимум 1 страница закрыта атрибутом** [Authorize(Roles = "Admin")] — например, админ-панель со списком пользователей или модерация контента.

- Неавторизованный пользователь, пытающийся открыть защищённую страницу, перенаправляется на логин. Авторизованный, но без нужной роли — получает 403 Forbidden.

6.3. Политики и resource-based авторизация

Что должно быть реализовано:

- **Минимум 1 custom-политика** с собственным AuthorizationHandler. Пример: «Может редактировать только свои задачи», «Может удалять только свои комментарии».
- Политика проверяет владельца ресурса: загружает сущность из базы, сравнивает UserId сущности с ID текущего пользователя. Админ может всё.
- Реализуйте resource-based авторизацию через IAuthorizationService.AuthorizeAsync(User, resource, "PolicyName") минимум на 1 странице (например, редактирование).
- В списке элементов: кнопки «Редактировать» и «Удалить» показываются только если пользователь имеет право (проверка через IAuthorizationService в Razor Page).

6.4. JWT для REST API

Что должно быть реализовано:

- Настройте две схемы аутентификации: Cookie (для Razor Pages) и JWT (для API-контроллеров). Укажите JWT как схему по умолчанию для API.
- Секретный ключ хранится в appsettings.json (длина — не менее 32 символов).
- POST /api/auth/login выпускает JWT с claims: ID пользователя, e-mail, роли. Срок жизни — не более 1 часа.
- Все API-эндпоинты защищены [Authorize]. Без токена — 401.
- При создании данных через API — UserId берётся из claims токена, а не из тела запроса.
- Обычный пользователь через API видит только свои данные. Админ — все.
- Swagger настроен для передачи JWT-токена.

6.5. Защита от CSRF

Что должно быть реализовано:

- Включите глобальный антифоргери-фильтр для всех POST-форм через AutoValidateAntiforgeryTokenAttribute.
- Убедитесь, что все формы на страницах (создание, редактирование, логин, регистрация, logout) используют Razor Tag Helpers — токен добавляется автоматически.
- В AJAX-запросах (POST/PUT/DELETE) антифоргери-токен передаётся в заголовке X-CSRF-TOKEN. Токен берётся из <meta name="csrf-token"> на странице. Сервер настроен читать токен из этого заголовка.
- Проверьте: AJAX-запрос без заголовка X-CSRF-TOKEN → 400 Bad Request.

6.6. Защита от XSS и security headers

Что должно быть реализовано:

- **Content Security Policy**: добавьте CSP-заголовок. Скрипты только с 'self' и разрешённых CDN (если используете). Инлайн-скрипты запрещены. Стили — только 'self'.
- **Минимум 3 security headers** из списка:
  - X-Content-Type-Options: nosniff
  - X-Frame-Options: DENY
  - Referrer-Policy: strict-origin-when-cross-origin
  - Strict-Transport-Security: max-age=31536000; includeSubDomains
- Проверьте: если в описании задачи/комментария написать <script>alert('XSS')</script> и сохранить — при выводе на странице скрипт не выполняется (Razor кодирует автоматически). CSP блокирует инлайн-скрипты даже при ошибке.

Что демонстрирует этот блок:
Вы умеете настраивать аутентификацию, авторизацию по ролям и политикам, защищать API через JWT, предотвращать CSRF и XSS, настраивать security headers.

Как технологии взаимодействуют

Технологии не существуют по отдельности. Они работают вместе.
Типичный сценарий в проекте:

1. **Пользователь регистрируется и входит** (Identity) — создаётся cookie, в навигации появляются защищённые ссылки.
2. **Пользователь открывает страницу** (Razor Page) — данные рендерятся на сервере из базы. Кнопки редактирования показываются только если политика разрешает (resource-based авторизация).
3. **Пользователь взаимодействует** — нажимает «лайк», меняет статус, добавляет комментарий (AJAX-запрос с антифоргери-токеном в заголовке).
4. **Сервер обрабатывает действие** — проверяет авторизацию, сохраняет в базу (EF Core) и отправляет уведомление через SignalR.
5. **Другие пользователи видят изменение мгновенно** — без перезагрузки, через SignalR. Хаб защищён [Authorize], данные отправляются только в нужную группу.
6. **Внешний клиент** (мобильное приложение, скрипт) работает с теми же данными через REST API, передавая JWT в заголовке. Без токена — 401, с токелем обычного пользователя — только свои данные.

Этапы работы и сроки

Этап	Что делается	Срок
1. Согласование темы	Описать предметную область, список страниц, сущностей, событий и ролей	Неделя 0
2. Проектирование	Схема БД (с ролями и пользователями), список страниц, API-эндпоинтов, SignalR-событий, политик	Неделя 0–1
3. База, Identity и Razor Pages	EF Core, Identity, миграции, seed (включая роли и админа), базовые страницы с серверным рендерингом и формами	Неделя 1–2
4. AJAX, REST API и JWT	Частичные обновления с антифоргери-токенами, API-контроллер, DTO, версионирование, JWT, Swagger	Неделя 2–3
5. SignalR и безопасность	Хаб с [Authorize], группы, переподключение, IHubContext, политики, CSP, security headers	Неделя 3
6. Подготовка к защите	Проверка, .http-файл, скриншоты, текст презентации	Неделя 3

Что сдавать

1. **Исходный код** проекта в репозитории (GitHub/GitLab).
2. **.http-файл** с запросами ко всем эндпоинтам REST API, включая логин (получение JWT) и защищённые вызовы с токеном.
3. **Swagger** доступен по адресу /swagger при запуске, с возможностью передать JWT.
4. **README.md** в корне репозитория: краткое описание проекта, инструкция запуска, список реализованных функций, логин и пароль seed-админа.
5. **Презентация** (5 минут):
    - тема и предметная область;
    - какая страница за какую технологию отвечает (с указанием конкретных страниц и действий);
    - как реализована безопасность: roles, policies, JWT, CSRF, CSP;
    - живая демонстрация: две вкладки, SignalR-обновления, AJAX с антифоргери-токеном, Swagger с JWT, попытка открыть чужую задачу → 403.

Примерные вопросы для защиты

1. Покажите страницу, где данные рендерятся на сервере. Почему здесь Razor, а не AJAX?
2. Покажите действие, где используется AJAX. Почему здесь AJAX, а не SignalR? Где антифоргери-токен в этом запросе?
3. Покажите событие, где используется SignalR. Почему здесь SignalR, а не AJAX? Как хаб защищён от неавторизованных пользователей?
4. Покажите REST API в Swagger. Чем он отличается от AJAX-обработчиков на страницах? Как передаётся JWT?
5. Что произойдёт, если пользователь был офлайн в момент SignalR-события? Получит ли он его позже? Как это решается в проекте?
6. Какая связь между AJAX-действием и SignalR-уведомлением? Кто вызывает кого?
7. Зачем нужны две версии API? В чём отличие?
8. Покажите миграцию, где создаются таблицы. Где заданы индексы?
9. Покажите код Seed. Как вы проверяете, что данные загружены? Где создаются роли и админ?
10. Покажите Fluent API. Зачем вы задали именно эти настройки?
11. Покажите запрос с .Include(). Что происходит в SQL (JOIN)?
12. Покажите M:N связь. Как EF Core хранит эту связь в БД?
13. Покажите custom-политику. Как проверяется, что пользователь может редактировать только свои данные?
14. Что произойдёт, если в JWT изменить payload, но оставить старую подпись?
15. Почему антифоргери-токен в AJAX передаётся в заголовке, а не в теле?
16. Что блокирует CSP и почему этого недостаточно для полной защиты от XSS?
17. Почему logout — это POST, а не GET?
18. Что произойдёт с пользователями при перезапуске сервера, если не настроены persistent ключи шифрования?
19. Чем SignInManager.PasswordSignInAsync отличается от CheckPasswordSignInAsync? Где какой используется?
20. Покажите, где обычный пользователь пытается открыть чужую задачу. Что происходит?
